

Jenkins - Pipeline CI/CD

Utilité dans le projet

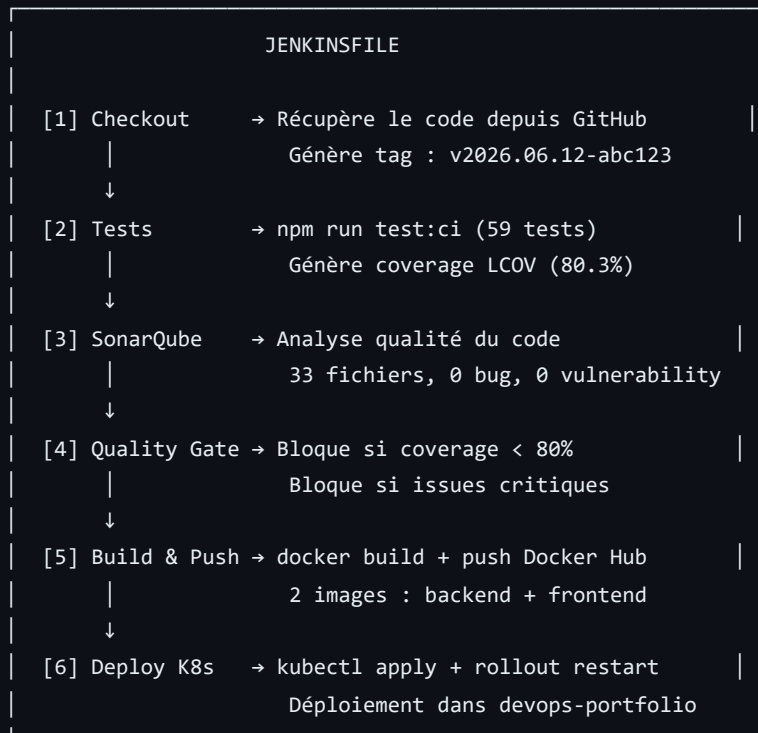
Jenkins automatise le **cycle de vie complet** du code : tests → qualité → build → déploiement.

Workflow automatisé

```
Git Push (main)
↓
Jenkins détecte le changement (webhook)
↓
Pipeline 6 stages s'exécute
↓
Application déployée sur Kubernetes
```

Pipeline à 6 stages

Architecture du pipeline





Détail des stages

Stage 1 : Checkout

```
stage('Checkout') {  
  steps {  
    checkout scm  
    script {  
      def shortCommit = sh(  
        script: 'git rev-parse --short HEAD',  
        returnStdout: true  
      ).trim()  
      env.IMAGE_TAG = "v${new Date().format('yyyy.MM.dd')}-${shortCommit}"  
    }  
  }  
}
```

Ce qui se passe :

- Clone le repository
- Génère un tag unique : `v2026.06.12-abc1234`
- Permet le rollback vers une version précise

Stage 2 : Backend Tests

```
stage('Backend Tests') {  
  tools {  
    nodejs 'NodeJS'  
  }  
  steps {  
    sh '''  
      cd backend  
      npm ci  
      npm run test:ci  
    '''  
  }  
}
```

Ce qui se passe :

- Installe les dépendances (`npm ci` = plus rapide que `npm install`)
- Lance Jest avec 59 tests
- Génère le rapport de coverage LCOV
- Échoue si des tests échouent

Stage 3 : SonarQube Analysis

```
stage('SonarQube Analysis') {
  steps {
    withSonarQubeEnv('SonarQube') {
      sh """
        sonar-scanner \
          -Dsonar.projectKey=devops-portfolio-mern \
          -Dsonar.sources=backend/src,frontend/src \
          -Dsonar.javascript.lcov.reportPaths=backend/coverage/lcov.info
      """
    }
  }
}
```

Métriques analysées :

- Bugs et vulnérabilités
- Code smells
- Duplications
- Coverage (doit être $\geq 80\%$)
- Complexité cyclomatique

Stage 4 : Quality Gate

```
stage('Quality Gate') {
  steps {
    timeout(time: 5, unit: 'MINUTES') {
      waitForQualityGate abortPipeline: true
    }
  }
}
```

Règles :

- ❌ Coverage < 80% → Pipeline échoue
- ❌ Bugs critiques → Pipeline échoue
- ❌ Vulnérabilités → Pipeline échoue
- ✅ Tous les critères OK → Continue

Stage 5 : Build & Push

```
stage('Build & Push') {
  steps {
    withCredentials([usernamePassword(...)]) {
      sh """
        docker login -u \${DOCKER_USER} -p \${DOCKER_PASS}

        docker build -t lims4/backend:latest ./backend
        docker build -t lims4/backend:\${IMAGE_TAG} ./backend
        docker push lims4/backend:latest
        docker push lims4/backend:\${IMAGE_TAG}
      """
    }
  }
}
```

2 tags par image :

- `latest` : Dernière version stable
- `v2026.06.12-abc1234` : Version spécifique (rollback)

Stage 6 : Deploy to Kubernetes

```
stage('Deploy to Kubernetes') {
  steps {
    sh """
      kubectl apply -f k8s/
      kubectl rollout restart deployment/backend-deployment -n devops-portfolio
      kubectl rollout status deployment/backend-deployment -n devops-portfolio
    """
  }
}
```

Ce qui se passe :

- Applique les manifests Kubernetes
- Force le redémarrage des pods
- Attend que le rollout soit terminé
- Vérifie que les nouveaux pods sont healthy

Notifications

Email automatique

```
post {
    success {
        mail(
            to: 'seydinalimamoulayeyade@gmail.com',
            subject: "✅ Build #${env.BUILD_NUMBER} - SUCCESS",
            body: "Pipeline terminé avec succès"
        )
    }
    failure {
        mail(
            to: 'seydinalimamoulayeyade@gmail.com',
            subject: "❌ Build #${env.BUILD_NUMBER} - FAILURE",
            body: "Pipeline échoué - intervention requise"
        )
    }
}
```

Déclenchement automatique

GitHub Webhook

```
GitHub (push sur main)
↓
Webhook → http://nuclei-mosaic-ecard.ngrok-free.app/github-webhook/
↓
Jenkins déclenche le pipeline
```

Configuration :

1. GitHub : Settings → Webhooks → Add webhook
2. Payload URL : Jenkins webhook URL (via ngrok)
3. Events : **push**

Poll SCM (fallback)

```
triggers {
    pollSCM('H/5 * * * *') // Vérifie toutes les 5 minutes
}
```

Credentials dans Jenkins

Secrets stockés

1. **dockerhub-credentials** : Username + Password Docker Hub

2. **jwt-secret** : Secret JWT pour l'application
3. **github-token** : Token pour accès privé (si besoin)

Utilisation sécurisée

```
withCredentials([
    usernamePassword(
        credentialsId: 'dockerhub-credentials',
        usernameVariable: 'DOCKER_USER',
        passwordVariable: 'DOCKER_PASS'
    )
]) {
    // Les variables sont masquées dans les logs
}
```

Métriques du pipeline

Métrique	Valeur
Durée moyenne	~5 minutes
Nombre de stages	6
Tests exécutés	59
Coverage	80.3%
Images buildées	2 (backend + frontend)
Déploiements	Kubernetes automatique

Gestion des échecs

Si tests échouent (Stage 2)

1. Pipeline s'arrête
2. Email envoyé
3. Logs disponibles dans Jenkins
4. Correction → nouveau commit → nouveau pipeline

Si Quality Gate échoue (Stage 4)

1. Pipeline s'arrête
2. Consulter SonarQube pour détails
3. Corriger les issues

4. Nouveau commit

Si déploiement échoue (Stage 6)

1. Pipeline s'arrête
2. Kubernetes garde l'ancienne version (pas de downtime)
3. Vérifier `kubect1 get pods -n devops-portfolio`
4. Rollback possible avec les images taguées

Rollback

Revenir à une version précédente

```
# Trouver le tag dans Docker Hub
docker pull lims4/backend:v2026.06.11-xyz789

# Mettre à jour le manifest K8s
kubect1 set image deployment/backend-deployment \
  backend=lims4/backend:v2026.06.11-xyz789 \
  -n devops-portfolio
```

Configuration Jenkins

Plugins requis

-  Docker Pipeline
-  Kubernetes CLI
-  SonarQube Scanner
-  NodeJS Plugin
-  Email Extension

Tools configurés

- **NodeJS** : Node.js 22
- **SonarScanner** : Pour l'analyse de code
- **Docker** : Pour build et push

Améliorations futures

1. **Tests parallèles** : Backend + Frontend en parallèle
2. **Cache Docker** : Réduire le temps de build
3. **Stage environment** : Déploiement staging avant prod

4. **Approval gate** : Validation manuelle avant prod

5. **Slack notifications** : Alertes en temps réel

Prochaine étape : [03-SONARQUBE.md](#)